1. Qué cambió y qué usar en cada pantalla

Esta guía explica cómo leer los datos del robot desde las aplicaciones Web y Móvil. La duda que surgió fue si, por la cantidad de sensores, había que hacer muchas consultas para llenar una sola pantalla. La respuesta es no: la pantalla principal se llena con una sola petición, y si se quiere que se actualice sola, se usa un WebSocket que avisa cuando el robot manda datos nuevos.

Para lograrlo se agregó a la base de datos una vista nueva, v_robot_telemetry_live, y se activó Supabase Realtime en las tablas telemetry_frames y alerts. No se modificó ninguna tabla ni la función de ingesta, y las cuatro vistas que ya existían siguen funcionando igual.

1.1 Qué consultar según lo que se quiere mostrar

Lo que se quiere mostrar	Qué consultar	Peticiones
Pantalla principal del robot en vivo	v_robot_telemetry_live + Realtime	1 por cada dato nuevo del robot
Abrir la pantalla o refrescar a mano	v_robot_telemetry_live	1
Lista de robots del domicilio	v_robot_telemetry_live sin filtro (una fila por robot)	1
Gráficas históricas de un sensor	v_readings_interpreted filtrada por robot, canal y fechas	1 por gráfica
Bandeja de alertas activas	Campo active_alerts del mismo paquete	0 adicionales
Reconocer una alerta (solo técnico)	rpc('ack_alert', { p_alert_id })	1

Por qué no se regresa el JSON del robot tal cual
El JSON original sí se guarda completo (columna raw_payload de telemetry_frames), pero trae los gases como lectura cruda del ADC. Si se entregara así, cada app tendría que programar la conversión a ppm, los umbrales y las alertas, y Web y Móvil podrían terminar mostrando números distintos. La vista entrega el mismo paquete ya interpretado, con la misma forma general del JSON de entrada (un arreglo readings con los 12 sensores).

2. Opción A: todo el estado del robot en una sola petición

Una consulta a v_robot_telemetry_live regresa una fila por robot con todo lo que necesita la pantalla principal: conexión, pose 2D, batería, estado, los 12 sensores y las alertas activas.

Web — supabase-js

```
// Web (supabase-js). La app usa la anon key y la sesión del usuario, nunca la service_role.
const { data: paquete, error } = await supabase
  .from('v_robot_telemetry_live')
  .select('*')
  .eq('robot_id', robotId)
  .maybeSingle();   // null si el usuario no tiene acceso a ese robot

if (error) console.error(error.message);
else if (paquete) pintarPantalla(paquete);
```

Móvil — Flutter

```
// Móvil (Flutter, supabase_flutter 2.x)
final paquete = await Supabase.instance.client
    .from('v_robot_telemetry_live')
    .select()
    .eq('robot_id', robotId)
    .maybeSingle();

if (paquete != null) pintarPantalla(paquete);
```

2.1 Cómo se ve la respuesta

Esta es la respuesta real que regresó la vista durante la prueba (se recortaron 9 de los 12 sensores). El archivo completo está en ejemplos/respuesta_v_robot_telemetry_live.json. El orden de las llaves puede variar; los nombres no.

Respuesta de v_robot_telemetry_live


```
{
  "robot_id": "22222222-2222-4222-8222-222222222222",
  "public_code": "CENTINELA-01",
  "home_id": "11111111-1111-4111-8111-111111111111",
  "frame_id": "9ed7255e-5874-4097-85b6-cdd21ec95e92",
  "stream_version": 18,
  "captured_at": "2026-10-23T00:06:10.127Z",
  "received_at": "2026-10-23T00:06:10.127Z",
  "seconds_since_last": 0,
  "connection": "online",
  "firmware_version": "0.2.0",
  "pose": {
    "map_version_id": "ecb3df81-477f-4e72-92c6-8b38f856cdb5",
    "x_m": 2.15,
    "y_m": 1.4,
    "theta_rad": 0.28,
    "sigma_position_m": 0.08,
    "sigma_heading_rad": null,
    "quality": "estimated"
  },
  "battery": {
    "voltage_v": 11.4,
```

```
    "current_a": null,
    "soc_pct": null,
    "method_code": "voltage_only",
    "quality": "voltage_only"
  },
  "state": {
    "reported": "moving",
    "navigation": "unknown",
    "motors_active": true,
    "wifi_rssi_dbm": null
  },
  "open_alerts": 1,
  "worst_severity": "critical",
  "readings": [
    {
      "channel_key": "mq6_raw",
      "metric_type": "gas_raw",
      "display_value": 2508.6,
      "display_unit": "ppm",
      "signal_pct": 85.5,
      "level": "critical",
      "confidence": "medium",
      "boolean_value": null,
      "raw_value": 3500,
      "raw_quality": "uncalibrated",
      "calibration_status": "field_clean_air",
      "gas_label": "LP (propano/butano)"
    },
    {
      "channel_key": "temperature",
      "metric_type": "temperature",
      "display_value": 24.6,
      "display_unit": "celsius",
      "signal_pct": null,
      "level": null,
      "confidence": "high",
      "boolean_value": null,
      "raw_value": 24.6,
      "raw_quality": "valid",
      "calibration_status": null,
      "gas_label": null
    },
    {
      "channel_key": "ir_floor_front_left",
      "metric_type": "digital",
      "display_value": null,
      "display_unit": "bool",
      "signal_pct": null,
      "level": null,
      "confidence": "high",
      "boolean_value": true,
      "raw_value": null,
      "raw_quality": "valid",
      "calibration_status": null,
      "gas_label": null
    }
    // ... 9 sensores más con la misma forma
  ],
  "active_alerts": [
    {
      "id": "5a756e70-6f17-43f9-b51a-7d19a61fc055",
      "type": "gas_mq6",
      "severity": "critical",
      "state": "acknowledged",
      "origin": "backend_rule",
```

```
      "opened_at": "2026-10-22T18:06:09.802-06:00",
      "last_seen_at": "2026-10-22T18:06:10.127-06:00",
      "details": {
        "gas": "LP (propano/butano)",
        "raw_adc": 3500,
        "crit_ppm": 2100,
        "warn_ppm": 1000,
        "ppm_estimated": 2508.6,
        "calibration_status": "field_clean_air"
      }
    }
  ]
}
```


2.2 Campos del paquete

Campo	Tipo	Para qué sirve
robot_id / public_code / home_id	UUID / texto / UUID	Identifican al robot y a su domicilio. Filtren siempre por robot_id.
frame_id / stream_version	UUID / entero	Último paquete recibido. stream_version sube de uno en uno; sirve para no repintar lo mismo.
captured_at / received_at	Fecha ISO (UTC)	Hora en que el robot tomó la lectura y hora en que llegó a la nube.
seconds_since_last / connection	Número / texto	Antigüedad del último paquete al momento de la consulta: online (< 10 s), stale (< 60 s), offline o never.
firmware_version	Texto	Versión del firmware del ESP32.
pose	Objeto o null	x_m, y_m, theta_rad, sigma_position_m, sigma_heading_rad, quality y map_version_id.
battery	Objeto o null	voltage_v, current_a, soc_pct, method_code y quality. Hoy solo se mide voltaje.
state	Objeto o null	reported (estado del firmware), navigation, motors_active y wifi_rssi_dbm.

Campo	Tipo	Para qué sirve
open_alerts / worst_severity	Entero / texto	Cuántas alertas siguen abiertas o reconocidas y la más grave (info, warning, critical).
readings	Arreglo (12)	Los 12 sensores del último paquete, ya interpretados. Ver tabla siguiente.
active_alerts	Arreglo	Alertas en estado open o acknowledged, de la más reciente a la más antigua.

Campos de cada elemento de readings

Campo	Para qué sirve
channel_key / metric_type	Qué sensor es (mq6_raw, temperature, ir_floor_front_left...) y de qué tipo.
display_value / display_unit	El valor que se muestra en pantalla. En gases son ppm estimados; en el resto es el valor medido con su unidad.
signal_pct	Solo gases: señal relativa del sensor de 0 a 100 %. Siempre viene, aunque no haya ppm.
level	Solo gases con calibración: normal, warning o critical.
confidence	Qué tan confiable es el dato: high, medium, uncalibrated o not_installed.
boolean_value	Solo sensores IR: true = condición segura, false = peligro u obstáculo.
raw_value / raw_quality	Lo que mandó el ESP32 tal cual. Útil para depurar; no se muestra al usuario.
calibration_status / gas_label	Solo gases: estado de la calibración y nombre del gas.

3. Opción B: actualización automática con WebSocket

Con Realtime la app no tiene que estar preguntando si hay datos nuevos; Supabase le avisa. El aviso se usa solo como señal para hacer la consulta de la opción A, de modo que siempre se pinta el paquete ya interpretado.

3.1 Cómo funciona

- **1.** Al entrar a la pantalla, la app hace la carga inicial con una petición a v_robot_telemetry_live y abre un canal de Realtime filtrado por robot_id.
- **2.** El robot manda su JSON cada 2 segundos. La función ingest_telemetry lo guarda completo (sensores, pose, batería y alertas) dentro de una sola transacción.
- **3.** Cuando esa transacción termina, Supabase avisa por el socket que hubo un INSERT en telemetry_frames para ese robot. El aviso llega después de que todo quedó guardado, así que la siguiente consulta ya trae el paquete completo.
- **4.** La app recibe el aviso y vuelve a pedir v_robot_telemetry_live una sola vez. Con eso repinta la pantalla.
- **5.** Si una alerta cambia (se abre, la reconoce un técnico o se resuelve), llega un aviso por la tabla alerts y se hace lo mismo.

Por qué no se pinta directo lo que llega por el socket
El aviso de Realtime trae la fila de telemetry_frames, incluido el JSON crudo del robot, pero sin interpretar: los gases vienen en ADC, sin ppm, sin nivel y sin alertas. Por eso se usa solo para saber que hay algo nuevo y enseguida se consulta la vista.

3.2 Código para Web

Web — supabase-js con Realtime

```
// Web (supabase-js): carga inicial + actualización automática por WebSocket
let versionPintada = null;

async function cargarPaquete(robotId, forzar = false) {
  const { data, error } = await supabase
    .from('v_robot_telemetry_live')
    .select('*')
    .eq('robot_id', robotId)
    .maybeSingle();
  if (error || !data) return;
  if (!forzar && data.stream_version === versionPintada) return;  // ya estaba en pantalla
  versionPintada = data.stream_version;
  pintarPantalla(data);
}

const canal = supabase
  .channel(`robot-${robotId}`)
  .on('postgres_changes',
      { event: 'INSERT', schema: 'public', table: 'telemetry_frames', filter: `robot_id=eq.${robotId}` },
      () => cargarPaquete(robotId))
  .on('postgres_changes',
      { event: '*', schema: 'public', table: 'alerts', filter: `robot_id=eq.${robotId}` },
      () => cargarPaquete(robotId, true))   // una alerta no cambia stream_version
  .subscribe((status) => {
    // Al conectar y cada vez que se reconecta, ponerse al día con una consulta
    if (status === 'SUBSCRIBED') cargarPaquete(robotId, true);
  });

// Al salir de la pantalla
supabase.removeChannel(canal);
```

3.3 Código para Móvil

Móvil — Flutter con Realtime

```
// Móvil (Flutter, supabase_flutter 2.x)
final supabase = Supabase.instance.client;

Future<void> cargarPaquete() async {
  final data = await supabase
      .from('v_robot_telemetry_live')
      .select()
      .eq('robot_id', robotId)
      .maybeSingle();
  if (data != null) pintarPantalla(data);
}

final filtro = PostgresChangeFilter(
  type: PostgresChangeFilterType.eq,
  column: 'robot_id',
  value: robotId,
);

final canal = supabase
    .channel('robot-$robotId')
    .onPostgresChanges(
      event: PostgresChangeEvent.insert,
      schema: 'public',
      table: 'telemetry_frames',
      filter: filtro,
      callback: (_) => cargarPaquete(),
    )
    .onPostgresChanges(
      event: PostgresChangeEvent.all,
      schema: 'public',
      table: 'alerts',
      filter: filtro,
      callback: (_) => cargarPaquete(),
    )
    .subscribe((status, _) {
      if (status == RealtimeSubscribeStatus.subscribed) cargarPaquete();
    });

// En dispose() de la pantalla
await supabase.removeChannel(canal);
```

4. Detalles que conviene cuidar

▪ **Robot apagado o sin WiFi.** Si el robot deja de mandar datos, el socket ya no avisa nada y la pantalla se queda con el último paquete. El campo connection se calcula en el momento de la consulta, así que no cambia solo. Recomendación: un temporizador local cada pocos segundos que compare la hora actual con received_at y use los mismos cortes que la vista (online menos de 10 s, stale menos de 60 s, offline después).

▪ **Nada de consultar cada 2 segundos.** Con Realtime no hace falta hacer polling. Solo se consulta al abrir la pantalla, al recibir un aviso y al reconectar.

- **Reconexión.** Si el teléfono pierde señal, el cliente de Supabase reconecta solo. Al volver al estado SUBSCRIBED hagan una consulta para recuperar lo que llegó mientras estaban desconectados (ya viene así en los ejemplos).
- **No repintar de más.** Si stream_version es igual al que ya está en pantalla, no hay nada nuevo. Ojo: cuando cambia una alerta, stream_version no sube, por eso en ese caso se fuerza el repintado.
- **Cerrar el canal.** Al salir de la pantalla llamen a removeChannel. Cada canal abierto es una conexión que sigue recibiendo eventos.
- **Sesión antes del socket.** Realtime respeta las mismas reglas de acceso (RLS) que las consultas. El usuario tiene que haber iniciado sesión antes de suscribirse; si no, no recibirá eventos.
- **Llaves.** Web y Móvil usan la anon key con la sesión del usuario. La service_role solo la usa el puente MQTT en el servidor; nunca debe ir en una app.
- **Fechas.** Todas las fechas vienen en UTC. Para mostrarlas, conviértanlas a la zona horaria del domicilio (homes.timezone, por defecto America/Mexico_City).

5. Cómo mostrar cada dato

Las reglas son las mismas de la documentación técnica; aquí se repiten en corto para tenerlas a la mano.

Caso	Qué llega	Qué mostrar
Gas con confidence = uncalibrated	display_value = null, signal_pct con valor	Barra con 'Señal relativa: XX %' y la leyenda 'sin calibrar'. No mostrar 0 ppm.
Gas con confidence = medium o high	display_value en ppm y level	Valor en ppm con color según level: normal verde, warning ámbar, critical rojo. En medium agregar 'estimado'.
Gas con confidence = not_installed	raw_value y display_value en null	Tarjeta gris: 'Sensor no instalado'. Hoy aplica al MQ-7.
Batería (method_code = voltage_only)	voltage_v con valor, soc_pct = null	Mostrar el voltaje, por ejemplo '11.4 V'. No mostrar 0 % cuando soc_pct venga en null.
IR de piso (ir_floor_*)	boolean_value	true: piso firme (verde). false: vacío o escalón (rojo).
IR laterales (ir_lateral_*)	boolean_value	true: costado libre (verde). false: pared u obstáculo cerca (ámbar).
Pose 2D	pose.x_m, pose.y_m, pose.theta_rad	Pasar de metros a píxeles con el mapa: px = (x_m - origin_x) / resolution_m. Si el plano se dibuja con Y hacia abajo, invertir Y.

6. Instalación y pruebas

6.1 Cómo aplicarlo en Supabase

- **Base nueva:** correr CENTINELA_supabase_completo.sql en el SQL Editor, como antes. Ya incluye la vista y Realtime.
- **Base ya instalada:** basta con volver a correr sql/04_vistas.sql. Es seguro repetirlo; no borra datos.
- **Verificar:** en Database > Publications, la publicación supabase_realtime debe listar telemetry_frames y alerts.

6.2 Qué se probó

La prueba automática (pruebas/run_test.mjs) corre sobre un PostgreSQL real local y terminó con 58 de 58 verificaciones correctas: las 42 que ya existían más 16 nuevas para esta parte.

- **Una sola fila por robot** que apunta al último paquete recibido (frame_id y stream_version).
- **Los 12 sensores** dentro de readings, con exactamente los mismos valores que v_latest_readings.
- **Pose, batería y estado** completos: x 2.15 m, y 1.40 m, 11.4 V con soc_pct en null, motores activos.
- **Gases** MQ-6 calibrado con ppm y nivel; MQ-7 como not_installed; IR de piso como booleano.
- **Alertas** una lectura alta de gas aparece como critical y la alerta reconocida viaja en active_alerts sin duplicarse.
- **Seguridad** un usuario del domicilio recibe su paquete; un usuario sin membresía no recibe nada.
- **Realtime** telemetry_frames y alerts quedan registradas en la publicación supabase_realtime.
- **Reinstalación** el script se puede correr varias veces sin error.

Pendiente de validar en el proyecto real
La prueba local confirma que la vista regresa el paquete correcto y que las tablas quedan registradas para Realtime. La conexión del WebSocket en sí depende del servidor de Supabase, así que conviene hacer una prueba rápida al montarlo: abrir la app, enviar un JSON con pruebas/enviar_a_supabase.mjs y ver que la pantalla se actualice.